The coherence reviewer

2026-08-16. Reflects `runner/turn/review.py`, `runner/turn/conductor.py` (`_review_and_redispatch`), and `agent_factories/general_specialist.py`.

A second opinion the orchestrator cannot skip. It runs in SERVER code, not as a tool the orchestrator chooses to call, and it exists for one failure: two specialists each answer correctly about a DIFFERENT window, and the synthesis reads as one coherent story.

(Diagrams are plain ASCII on purpose: box-drawing glyphs are outside Courier's WinAnsi encoding and render substituted or blank in the PDF.)

Where it sits in the turn

```
screen -> cache-replay -> assemble -> ORCHESTRATOR (phase 1)
                                          |
                                          v
                            +---- cancel checkpoint ----+
                                          |
                                  REVIEW + REDISPATCH        <-- this doc
                                          |
                            +---- cancel checkpoint ----+
                                          |
                                  ensure report_agent
                                          |
                                      finalize
```

It is deliberately AFTER the orchestrator has produced a FinalAnswer. The answer already exists; the reviewer decides whether to let it stand.

The gate: two specialists or nothing

```
_is_multi_specialist_turn(ctx)
     |
     +-- fewer than 2 DISTINCT domain specialists --> SKIP ENTIRELY
     |      no reviewer, no extra Runner run, zero added latency
     |      (a single specialist has nothing to be incoherent WITH)
     |
     +-- 2 or more --> run the reviewer
```

`general_specialist` and `report_agent` are aux tools: they never count toward the gate and their payloads are never fed to the reviewer. So the reviewer cannot review itself.

The flow

```
tool_calls (phase 1)
     |
     |  specialist_outputs = {tool: payload}   aux tools excluded
     v
emit agent_started (general_specialist)   <- trace shows it BEFORE the verdict
     |
     v
_run_review  ---- Runner.run(reviewer, {question, specialist_outputs}) ----+
     |                          asyncio.wait_for(120s)                     |
     |                          node: general_specialist, tag: "review"    |
     |                                                                     |
     |  ANY exception, timeout included, is swallowed -----> returns None  |
     |  logs review_failed                                                 |
     v                                                                     |
```

```
ReviewReport <---------------------------------------------------------- -+
     |  resolved[] / open_conflicts[] / cross_domain_insights[]
     |  directive: {kind, specialist, anchor, why}
     v
emit agent_completed + trace payload, log review_done
     |
     +-- kind == "coherent" -----------------> ship phase-1 answer unchanged
     |
     +-- kind == "qualified_release" --------> ship it too (INERT today,
     |                                          see Limits)
     |
     +-- kind == "needs_redispatch"
            |
            +-- _dispatch_count >= 2 --> CAPPED: ship, with a residual flag
            |                            log review_capped
            |
            +-- under the cap
                   |  bump _dispatch_count
                   |  invalidate that specialist's phase-1 distillation
                   |  append a [REVIEW DIRECTIVE] user turn to the
                   |  phase-1 transcript, then RESUME the orchestrator
                   v
                phase 3: orchestrator re-runs ONLY that specialist,
                anchored, and re-synthesizes -> new FinalAnswer
```

What the reviewer is handed, and what it can do

Input is a JSON blob: the framed question plus `{specialist: payload}` for every domain specialist that ran.

It carries **verification-only** data tools — `list_available_tables`, `get_table_schema`, `aggregate_column`, `batch_aggregate`, plus `make_chart` / `get_chart_guidance` for cross-domain comparison figures. That lets it re-measure a disputed number and name the canonical value.

`query_table`, `summarize_trend` and `summarize_by_group` are **pointedly excluded**: raw row dumps invite scope creep, and trend/group work is the specialists' job. The reviewer is there to check an aggregate, not to introduce new factual claims.

It **never dispatches**. The directive is advisory; the orchestrator acts on it. That separation is why a reviewer failure can only cost the correction, never the answer.

The directive

```
kind = coherent | needs_redispatch | qualified_release

needs_redispatch carries:
   specialist   who to re-run
   anchor       the window/event to anchor to, e.g. "2025-05"
   why          one line; feeds the sub-question AND the user-visible flag
```

The injected turn is literal, and the shape matters — it re-runs ONE specialist, not the team:

```
[REVIEW DIRECTIVE] needs_redispatch: re-invoke `<specialist>` anchored to
<anchor>. Reason: <why> Re-run ONLY that specialist with the anchor folded
into its sub-question, then synthesize the final answer.
```

Re-dispatch KB hygiene

Easy to miss, and it is why a correction does not leave a corrupted KB behind.

The re-dispatched specialist already distilled its phase-1 answer into knowledge points. Topic supersession does NOT cover this: a mis-anchored driver KP (`..._2024_09`) has a DIFFERENT topic from the corrected one

(`..._2025_05`), so both would stay active and the next turn would read two contradictory anchors.

So before the re-run, scoped strictly to that specialist and this turn:

```
1. cancel in-flight distill-<specialist> / autochart-<specialist> tasks
2. drop ctx._specialist_kb[<specialist>] entries with captured_at_turn == turn
```

Other specialists' KPs and earlier turns' KPs are untouched. Best-effort — a hygiene failure must not block the correction itself.

Every failure degrades to the answer we already have

Three nested guards, all deliberate:

layer	on failure	cost
`_run_review`	swallows ALL exceptions incl. timeout, logs `review_failed`, returns `None`	the coherence gate is skipped for this turn
`_apply_review_directive`	outer try/except, logs `review_phase_error`	as above, plus the re-dispatch
`_review_and_redispatch`	phase-1 `final_answer` stands	nothing else

The turn must NEVER block on the reviewer. The design cost of that: **a review that silently fails is indistinguishable from one that found nothing** — both ship the phase-1 answer with no flag. `review_done` vs `review_failed` in the JSONL is the only way to tell them apart.

Its own fence, since 2026-08-16

The reviewer used to share `ORCH_PLAN_TIMEOUT_S` (25s) with the round-1 planning watchdog, and the two want opposite treatment:

```
                   on expiry                      so it wants
-----------------  -----------------------------  --------------------
round-1 watchdog   replan; deadline DISARMED on   25s TIGHT
                   the final attempt              (it retries)
reviewer           returns None; the gate is just room to OUTLAST a
                   dropped for the turn           stall (no retry)
```

At 25s it also sat under the 40s per-call stall fence, so the call-layer retry was unreachable for it — the phase fence always fired first. That is the private-env failure where `general_specialist` died at exactly 25.00s while `round_1` had run 0.00s. Now `reviewer_s = 120`, sized to hold one worst-case call (40s stall fence + 60s retry budget = 100) plus its own work. See `timeouts-and-retries` for the full layer map.

What it looks like in the trace

```
agent_started      general_specialist          before the run
agent_completed    general_specialist          payload = review summary
node_trace         node "general_specialist", depth 0, tag "review"
JSONL              review_done {directive_kind, review_ran, n_domain_specialists}
                   review_redispatch {specialist, anchor, dispatch_count}
                   review_capped / review_failed / review_phase_error
flags (visible)    "coherence_review: re-dispatched `X` anchored to Y (why)"
                   "coherence_review: re-dispatch needed but the <=2 cap ..."
```

Both the re-dispatch and the cap surface a user-visible flag. A `coherent` verdict is silent — which is right, but it means the common case leaves no trace in the answer.

Limits

- `qualified_release` **is declared but not acted on.** The schema and the reviewer's skill both offer it; `_apply_review_directive` only branches on `needs_redispatch`, so it behaves exactly like `coherent`. It is the seam left for early qualified-release (Task 7), not a working path.
- **Silent failure looks like success.** `_run_review` returning `None` ships the phase-1 answer with no flag, identical to a `coherent` verdict. Dev shows 122 `review_done` / 0 `review_failed`, so this has not bitten here — but prod is where the 25.00s death happened, and it would have looked like agreement.
- **One correction round, and the cap counts the INITIAL dispatch.** `_dispatch_count` starts at 1 for the phase-1 dispatch, so `< 2` allows exactly one re-dispatch per turn.
- **The reviewer sees payloads, not tool calls.** It judges the specialists' stated findings and evidence, so a wrong number that both specialists agree on is invisible to it unless it re-measures with its own aggregate tools.
- **Single-specialist turns are never reviewed.** Correct by construction — nothing to cross-check — but it means the coherence gate does not exist on the cheapest, most common turns.